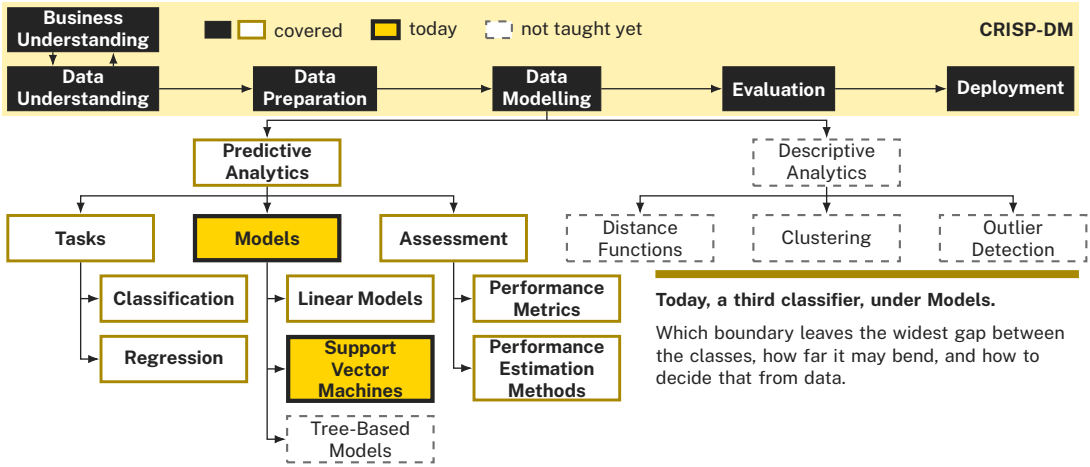
DALHOUSIE UNIVERSITY

Support Vector Machines

Margins, kernels, and tuning inside cross-validation

Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Where Support Vector Machines Sit



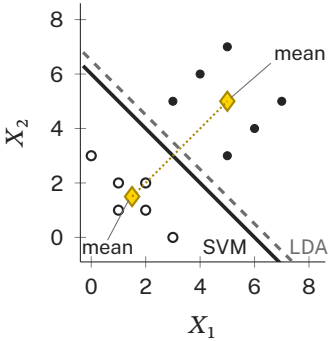
Today, a third classifier, under Models.

Which boundary leaves the widest gap between the classes, how far it may bend, and how to decide that from data.

How LDA Draws Its Line

1. LDA turns each case into one score, a weighted sum $z = w_1 X_1 + w_2 X_2$.
2. It picks the weights that push the two class **means** apart and keep each class **tight**, Fisher's separation score.
3. The boundary sits where a case is equally close to both means, so here it is $X_2 = 6.5 - X_1$, halfway between $(1.5, 1.5)$ and $(5, 5)$.

Every case helps place the line, through the two means and the shared spread.



The twelve cases used throughout today. Dashed, LDA. Solid, the SVM, coming next.

Same Data, a Different Question

LDA asks

Where do the classes sit on average, and how spread are they? Its line comes from the means and the shared spread, so it assumes each class is a roughly Normal cloud.

The SVM asks

Which line leaves the most empty room between the classes? Its line comes only from the cases nearest the gap, and it assumes nothing about the shape of each class.

- On these twelve cases both lines run in the same direction, and LDA's sits half a unit higher.
- Move a case far from the gap and LDA's line moves, because the mean moves. The SVM's does not.
- Move a case near the gap and the SVM's line moves at once. LDA's barely does.
- *k*NN, the third classifier so far, draws no line at all. It votes locally and can bend around anything.

Six Ideas, One per Problem

DALHOUSIE UNIVERSITY

An SVM starts from the widest straight street and bends only as far as two settings allow.

Idea	The problem it answers	What you set or check
Widest margin	many boundaries fit the training data	nothing; the fit chooses it
Soft margin	real classes overlap	the cost C
Kernel	the boundary is not straight	the kernel and its γ
Scaling	distances depend on units	a scaler fitted on training rows
Tuning	C and γ interact	a grid chosen inside, scored outside
Class weights	a rare class is ignored	weights and a class-aware metric

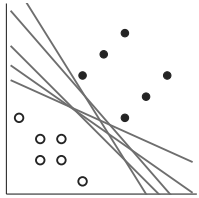
The Widest Street

DALHOUSIE UNIVERSITY

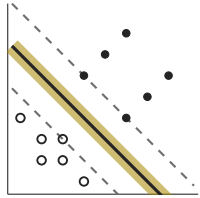
Of all the lines that separate two classes, take the one with the most room on both sides.

Which Separating Line?

infinitely many separators



the maximum-margin one

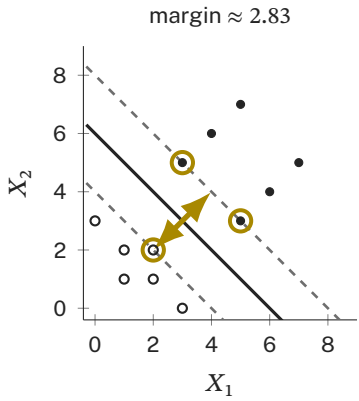


- Many lines separate these two classes, all with **zero training error**, so training error cannot choose.
- The SVM takes the line in the middle of the **widest street** between the classes.
- A wide street survives the largest shift in new cases before it errs.

The Margin and the Support Vectors

- The width of the street is the **margin**, 2.83 units here.
- The three ringed cases sit on its edges. They are the **support vectors**. Delete the other nine and the same line comes back.
- Every case gets a score, its signed distance from the line. The sign gives the class, and the score ranks cases for an ROC curve.

Far-away cases do not matter. Cases on the edge decide everything.



The Widest Street in R and Python

DALHOUSIE UNIVERSITY

R (`e1071`)

```r
library(e1071)
d <- data.frame(
  x1 = c(5, 3, 6, 4, 7, 5, 2, 1, 2, 1, 3, 0),
  x2 = c(3, 5, 4, 6, 5, 7, 2, 1, 1, 2, 0, 3),
  y  = factor(rep(c(1, -1), each = 6),
              levels = c(1, -1)))
s <- svm(y ~ x1 + x2, d, kernel = "linear",
         cost = 1e5, scale = FALSE)
s$index        # 1 2 7, the support vectors
w <- t(s$coefs) %*% s$SV       # (0.5, 0.5)
-s$rho         # b = -3
2 / sqrt(sum(w^2))             # margin 2.83
```

Python (`scikit-learn`)

```python
import numpy as np
from sklearn.svm import SVC

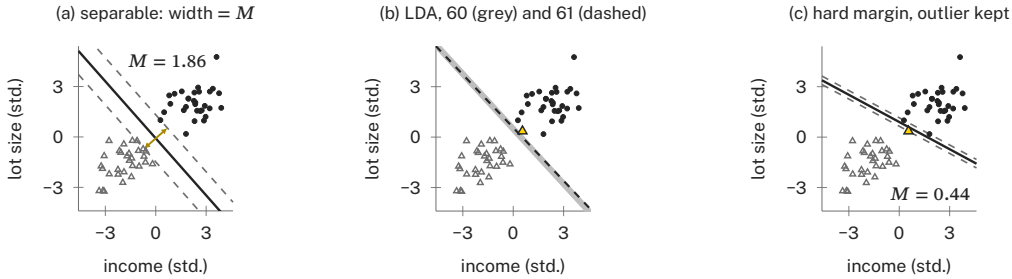
X = np.array([[5, 3], [3, 5], [6, 4], [4, 6],
              [7, 5], [5, 7], [2, 2], [1, 1],
              [2, 1], [1, 2], [3, 0], [0, 3]])
y = np.array([1] * 6 + [-1] * 6)
s = SVC(kernel="linear", C=1e5).fit(X, y)
print(s.support_)     # [6 0 1], zero-based row
  ↪ numbers
print(s.coef_, s.intercept_)        # (0.5, 0.5), -3
print(2 / np.linalg.norm(s.coef_))  # margin 2.83
```

A huge cost allows no case inside the street. Both libraries return the same three support vectors and the same width. R counts rows from 1, Python from 0.

Soft Margins

Real classes overlap. Let a few cases break the rule, and charge a fine for each one.

One Outlier Against a Strict Street

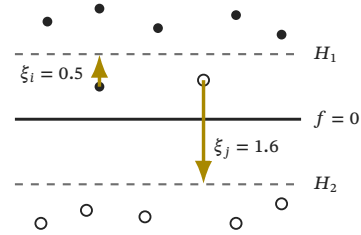


A street that must separate everything shrinks from 1.86 to 0.44 when **one** case lands on the wrong side, panel (c). LDA barely moves, panel (b), because its line comes from the class means.

After Boehmke and Greenwell, *Hands-On Machine Learning with R*, ch. 14.

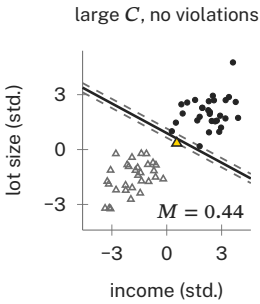
C Is the Fine per Violation

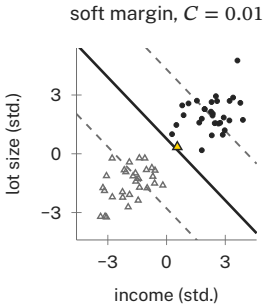
- A case may step into the street, or even cross the line, if it pays a fine that grows with how far it intrudes.
- The cost C sets the fine. The fit balances a wide street against the total fines.
- Every case on or inside the street is now a support vector, including the misclassified ones.



Each arrow is one case's intrusion past its own edge of the street.

The Two Ends of the Dial





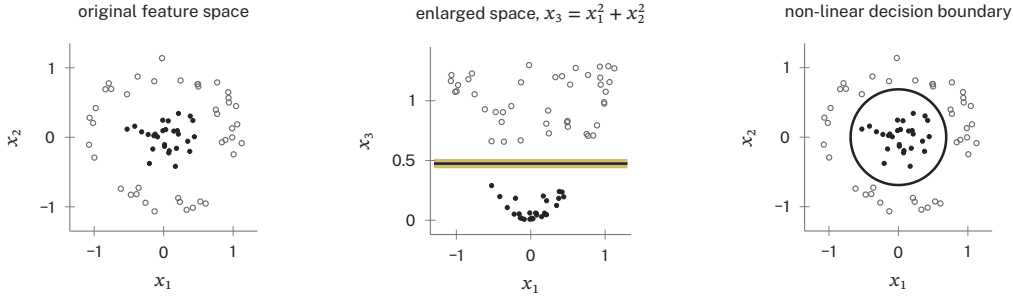
- **Large** C, fines are steep. A narrow street bends around the outlier. Risk of overfitting.
- **Small** C, fines are cheap. A wide street ignores the outlier, with 26 support vectors and 1 training error. Too small underfits.

Training error always prefers a larger C. Choose C by cross-validation, like K in kNN.

Kernels

When no straight line works, a new feature can make one work.

A Derived Feature Can Straighten a Boundary



No straight line separates a disc from a ring. Add the derived feature $x_1^2 + x_2^2$, the squared distance from the centre, as in feature engineering. Now a flat cut separates them, and read back on the original axes it is a circle.

After Boehmke and Greenwell, *Hands-On Machine Learning with R*, ch. 14.

A Kernel Is a Similarity

- Building every derived feature by hand is slow, and you would have to guess which ones help.
- The fit only ever needs one number per pair of cases, **how similar they are**. A **kernel** computes it from the original columns.
- A new case is classified by a vote of the support vectors, each weighted by how similar it is.

> Like kNN, except that only the support vectors vote, and the closer ones count for more.

The **radial** kernel, the default in both libraries, uses

$$\text{similarity} = e^{-\gamma \times (\text{squared distance})}$$

It is 1 for identical cases and falls towards 0 with distance.

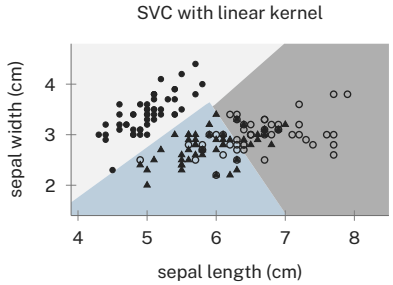
Gamma Is the Reach of One Case

DALHOUSIE UNIVERSITY

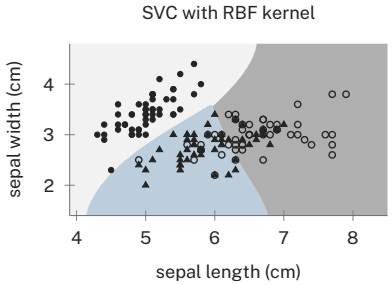
- The cases $(5, 3)$ and $(2, 2)$ are at squared distance 10. With $\gamma = 0.05$ their similarity is $e^{-0.5} \approx 0.61$, and with $\gamma = 0.5$ it is $e^{-5} \approx 0.007$.
- **Large** γ, each support vector reaches only a small island around itself. It overfits, like $K = 1$ in kNN.
- **Small** γ, every case looks alike and the boundary is nearly straight.

γ and C both buy flexibility, so one can stand in for the other. Tune them **together**.

Straight or Curved, the Same Algorithm



linear kernel



radial kernel

Two iris measurements, three species. Only the kernel changed. Neither boundary separates versicolor from virginica, which overlap in these two measurements.

After the scikit-learn gallery, *Plot different SVM classifiers in the iris dataset.*

The Workflow

C and gamma are settings, so choosing them is fitting. Nested validation already says how.

A New Unit Moves the Boundary

R

```r
d10 <- transform(d, x1 = 10 * x1)  # new unit
s10 <- svm(y ~ x1 + x2, d10, kernel = "linear",
           cost = 1e5, scale = FALSE)
s10$index      # 2 7 11, no longer 1 2 7
```

Python

```python
X10 = X * [10, 1]     # x1 in a new unit
s10 = SVC(kernel="linear", C=1e5).fit(X10, y)
print(np.sort(s10.support_))  # [ 1  6 10], not [0 1
  ↪ 6]
```

Record x_1 in a unit ten times smaller. The cases are the same, yet the support vectors and the line change, because the street is measured in the predictors' units. Standardize every predictor, fitted on the training rows only, as for kNN.

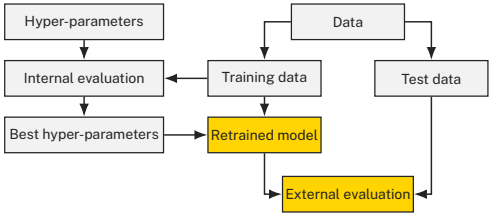
`e1071::svm` standardizes for you. `SVC` does not, so put a `StandardScaler` in front of it.

Tune Inside, Score Outside

`Glass`, 214 fragments, nine chemical measurements, six glass types of 9 to 76 fragments each.

1. Cut the rows into five stratified **outer** folds.
2. Inside each outer training set, choose C and γ by five **inner** folds.
3. Score the chosen model once on the outer fold it never saw.

Balanced accuracy, the mean recall over the six types, so the small types count as much as the large ones.



Nesting, the fix for a lucky search.

Reading the Grid

$C \setminus \gamma$	0.001	0.01	0.1	1
0.1	0.167	0.167	0.278	0.176
1	0.167	0.376	0.575	0.479
10	0.377	0.532	0.629	0.523
100	0.495	0.642	0.649	0.519
1000	0.598	**0.742**	0.660	0.519
10000	0.707	0.725	0.660	0.519

Inner-fold balanced accuracy, outer fold 1.

- C and γ grow in powers of ten, because both act by multiplying.
- **Top rows** are too stiff. 0.167 is one sixth, a model that answers one type for every case.
- **Along a diagonal** scores are similar, larger C with smaller γ buying the same flexibility.
- The best cell sat at the edge of a first grid, so the grid was widened to $C = 10000$.

The Nested Loop in R and Python

DALHOUSIE UNIVERSITY

R

```r
outer_score <- function(k) {
  d <- Glass[outer != k, ]
  inner <- createFolds(d$Type, 5, FALSE)
  cv <- function(g, C) mean(sapply(1:5,
    function(j) {
      m <- svm(Type ~ ., d[inner != j, ],
               gamma = g, cost = C)
      p <- predict(m, d[inner == j, ])
      bacc(d$Type[inner == j], p)
    }))          # choose INSIDE
  b <- grid[which.max(mapply(cv,
         grid$gamma, grid$cost)), ]
  m <- svm(Type ~ ., d, gamma = b$gamma, cost = b$cost)
  test <- Glass[outer == k, ]
  bacc(test$Type, predict(m, test))
}                # score OUTSIDE
scores <- sapply(1:5, outer_score)
round(scores, 3)
## 0.754 0.657 0.727 0.596 0.684
```

Python

```python
outer = []
for tr, te in folds.split(X, y):
    gs = GridSearchCV(svm, grid, cv=folds,    # INSIDE
        scoring="balanced_accuracy").fit(X[tr], y[tr])
    outer.append(balanced_accuracy_score(     # OUTSIDE
        y[te], gs.predict(X[te])))
print(np.round(outer, 3))
# [0.553 0.421 0.679 0.648 0.766]
```

Same Folds for Every Model

Four models, each tuned inside the training rows of every outer fold and scored on the same five outer folds, the folds the Python listing draws.

Outer fold	1	2	3	4	5
Baseline, most common class	0.167	0.167	0.167	0.167	0.167
LDA	0.574	0.613	0.452	0.608	0.481
kNN, K tuned	0.638	0.583	0.685	0.581	0.663
SVM, C and γ tuned	0.553	0.421	0.679	0.648	0.766
SVM minus kNN	−0.085	−0.162	−0.006	+0.067	+0.103

Means 0.167, 0.546, 0.630 and 0.613. The SVM wins two folds and loses three to kNN.

The corrected t-test, with $1/4$ for five folds, gives $p = 0.83$ for the SVM against kNN. The SVM, kNN and LDA cannot be told apart here, though all three beat the baseline.

Class Weights

- The fine C is the same for every class, so a type with nine fragments adds little to the total and the fit can afford to miss all of it.
- **Class weights** raise the fine for the small classes. Both libraries accept them, and `"balanced"` gives every class the same total weight.
- Treat weighting as one more setting, chosen inside like C and γ, and judge it with the class-aware metric it is meant to improve.

Plain accuracy can hide what weights do for a small class, so report balanced accuracy or the recall of each class.

Three Ideas to Keep

DALHOUSIE UNIVERSITY

The widest street

The SVM takes the boundary with the most room on both sides. The support vectors on its edges decide it.

A fine and a reach

C is the fine per violation. A kernel is a similarity, and γ is how far one case reaches.

The same workflow

Scale the predictors, tune C and γ inside, score outside, and test the gap to earlier models.

Next comes a classifier you can read as rules, the decision tree.